

THOTH CLOUD

Plateforme Cloud Multi-Services

Rapport d'Implémentation Backend

TP4 : Évolution vers un Mini Cloud PaaS

INF4107

Virtualisation et Cloud Computing

Master 1 - Systèmes & Réseaux

Enseignants : Valery MONTHE & Gildas NGOUANFO

Année : 2025-2026

Table des matières

1	Introduction	4
1.1	Objectifs du Projet	4
1.2	Lien GitHub	4
2	Architecture Globale	4
2.1	Vue d'Ensemble	4
2.2	Stack Technologique	5
3	Partie 1 : Amélioration du Service IaaS	5
3.1	Multi-Tenant avec Isolation Utilisateur	5
3.1.1	Système de Nommage des VMs	5
3.1.2	Filtrage par Utilisateur	5
3.2	Découplage de l'Application	6
3.2.1	Remote Executor	6
3.3	Isolation Réseau	7
3.3.1	Network Manager	7
3.4	Flavors et Gestion des Ressources	8
3.4.1	Définition des Flavors	8
3.4.2	Flavors par Défaut	9
3.4.3	Validation des Ressources	9
3.5	Gestion Multi-Hôtes	10
3.5.1	Stratégies de Sélection d'Hôte	10
3.5.2	Autres Stratégies Possibles	12
4	Partie 2 : Cluster Docker Swarm	12
4.1	Déploiement Automatique	12
4.1.1	Architecture du Cluster	12
4.1.2	Installation Docker via Cloud-Init	14
4.1.3	Initialisation du Swarm	14
4.1.4	Jonction des Workers	15
4.2	Exemple de Déploiement	16
5	Partie 3 : Service PaaS	16
5.1	Catalogue d'Applications	16
5.1.1	Liste des Applications	16
5.2	Mutualisation des Bases de Données	16
5.2.1	Architecture de Mutualisation	17
5.2.2	Création de Base de Données	17
5.2.3	Script de Gestion des Bases	18
5.3	Déploiement d'Applications	18
5.3.1	Exemple : WordPress	18
5.4	Système de Facturation	19
5.4.1	Calcul des Coûts	19
5.4.2	Exemple de Facturation	20

6	Partie 4 : Service SaaS	20
6.1	Plateforme Collaborative	20
6.1.1	Architecture SaaS	21
6.1.2	Déploiement Docker Compose	21
6.1.3	Intégration OwnCloud + OnlyOffice	22
6.2	Provisionnement d'Utilisateurs	23
6.3	Facturation Stockage	23
7	Documentation API	24
7.1	Authentification	24
7.1.1	Login	24
7.2	Gestion des VMs	24
7.2.1	Déployer une VM	24
7.2.2	Lister les VMs	25
7.3	Gestion Swarm	25
7.3.1	Créer un Cluster	26
7.4	Déploiement PaaS	26
8	Guide de Déploiement	27
8.1	Prérequis	27
8.2	Installation	27
8.2.1	1. Cloner le Repository	27
8.2.2	2. Setup KVM Host	27
8.2.3	3. Compiler le Backend	28
8.2.4	4. Configurer l'Environnement	28
8.2.5	5. Initialiser les Bases de Données	28
8.2.6	6. Démarrer le Backend	28
8.2.7	7. Créer un Utilisateur Admin	29
8.3	Multi-Hôtes (Optionnel)	29
8.4	Vérification	29
9	Limites et Améliorations	30
9.1	Limites Connues	30
9.2	Améliorations Futures	30
9.2.1	Court Terme (3 mois)	30
9.2.2	Moyen Terme (6 mois)	31
9.2.3	Long Terme (12 mois)	31
10	Conclusion	31
10.1	Objectifs Atteints	32
10.2	Compétences Développées	32
10.3	Perspectives	32
A	Structure du Code	33
A.1	Organisation des Fichiers	33

B	Configuration Système	33
B.1	Fichiers de Configuration	34
B.1.1	/var/lib/thoth-cloud/.env	34
B.1.2	/var/lib/thoth-cloud/hosts.json	34
C	Commandes Utiles	34
C.1	Gestion des VMs	34
C.2	Gestion Docker Swarm	35

1 Introduction

Contexte du Projet

THOTH CLOUD est une plateforme cloud complète implémentée en C++ qui fournit des services IaaS, PaaS et SaaS. Ce rapport documente l'architecture backend et les implémentations techniques réalisées conformément aux exigences du TP4.

1.1 Objectifs du Projet

Le projet vise à créer un cloud multi-tenant avec :

- **IaaS amélioré** : Multi-utilisateurs, isolation réseau, gestion multi-hôtes
- **PaaS** : Déploiement automatique de clusters Docker Swarm et applications
- **SaaS** : Plateforme collaborative (OwnCloud + OnlyOffice)
- **Facturation** : Système de billing basé sur la consommation

1.2 Lien GitHub

Dépôt du Projet

<https://github.com/votre-organisation/thoth-cloud>

2 Architecture Globale

2.1 Vue d'Ensemble

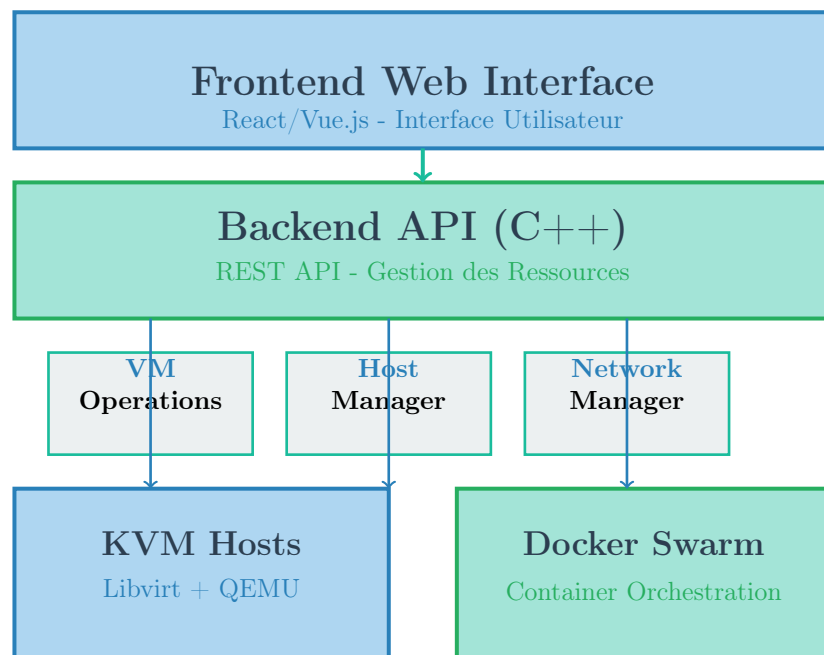


FIGURE 1 – Architecture en Couches de THOTH CLOUD

2.2 Stack Technologique

Composant	Technologie
Langage Backend	C++17
Framework HTTP	cpp-httplib
Sérialisation JSON	nlohmann/json
Virtualisation	Libvirt + QEMU/KVM
Conteneurisation	Docker + Docker Swarm
Orchestration	Cloud-init, Ansible
Base de données	PostgreSQL, MariaDB
Réseau	Libvirt networks, iptables

TABLE 1 – Technologies Utilisées

3 Partie 1 : Amélioration du Service IaaS

3.1 Multi-Tenant avec Isolation Utilisateur

Isolation des Ressources

Chaque utilisateur dispose de ressources isolées et ne peut accéder qu'à ses propres VMs, réseaux et clusters.

3.1.1 Système de Nommage des VMs

Implémentation dans `utils.hpp` - classe `VMNameManager` :

```

1 std::string createVMName(const std::string& username,
2                          const std::string& userVMName) {
3     std::string cleanUser = sanitize(username);
4     std::string cleanVMName = sanitize(userVMName);
5     std::string timestamp = getTimestamp();
6
7     // Format: username__vmname__timestamp
8     return cleanUser + SEPARATOR + cleanVMName + SEPARATOR +
9         timestamp;
10 }
```

Listing 1 – Génération de noms de VMs uniques

Avantages :

- Noms globalement uniques
- Identification rapide du propriétaire
- Impossible pour un utilisateur de deviner les noms des VMs d'autrui

3.1.2 Filtrage par Utilisateur

```

1 int UserOperations::listUserDomains(virDomainPtr **domains,
2                                   std::string username,
3                                   int flags) {
4     virDomainPtr* allDomains = nullptr;
5     int numDomains = virConnectListAllDomains(conn, &allDomains,
6                                               flags);
7
8     // Filter by username prefix
9     VMNameManager::filterUserVMs(allDomains, username,
10                                domains, numDomains,
11                                &numUserDomains);
12 return numUserDomains;
13 }

```

Listing 2 – Filtrage des VMs par utilisateur

3.2 Découplage de l'Application

Architecture Découplée

L'application backend est séparée des hôtes KVM et communique via SSH/libvirt remote.

3.2.1 Remote Executor

Classe RemoteExecutor (remote_executor.hpp/cpp) :

```

1 class RemoteExecutor {
2 private:
3     virConnectPtr conn;
4     bool isRemote;
5     std::string remoteUser;
6     std::string remoteHost;
7     std::string sshKeyFile;
8
9     std::string buildSSHCommand(const std::string& command) const {
10         if (!isRemote) return command;
11
12         std::stringstream ssh;
13         ssh << "ssh ";
14         if (!sshKeyFile.empty()) {
15             ssh << "-i " << sshKeyFile << " ";
16         }
17         ssh << "-o StrictHostKeyChecking=no ";
18         ssh << remoteUser << "@" << remoteHost << " ";
19         ssh << "'" << command << "'";
20         return ssh.str();
21     }
22
23 public:

```

```
24   ExecResult execute(const std::string& command) const;  
25   bool fileExists(const std::string& path) const;  
26   long long getAvailableDiskSpace(const std::string& path) const;  
27 };
```

Listing 3 – Exécution de commandes à distance

Fonctionnalités :

- Détection automatique du mode (local/remote)
- Authentification par clé SSH
- Vérification de fichiers distants
- Gestion de l'espace disque

3.3 Isolation Réseau

Stratégie d'Isolation

- Un réseau libvirt dédié par utilisateur
- Un réseau IP séparé par cluster Docker Swarm
- Attribution automatique de sous-réseaux

3.3.1 Network Manager

Implémentation dans `network_manager.hpp/cpp` :

```
1 json NetworkManager::createUserNetwork(const std::string& username,  
2                                         const std::string&  
3                                         networkName) {  
4  
5     json result;  
6  
7     // Check user network limit  
8     if (getUserNetworkCount(username) >= MAX_USER_NETWORKS) {  
9         result["error"] = "User network limit reached";  
10        return result;  
11    }  
12  
13    // Generate unique subnet  
14    std::string subnet = generateSubnet();  
15    std::string networkId = generateNetworkId();  
16  
17    // Create libvirt network XML  
18    std::string xml = generateUserNetworkXML(networkId,  
19                                             displayName,  
20                                             subnet);  
21  
22    // Define and start network  
23    virNetworkPtr network = virNetworkDefineXML(conn, xml.c_str());  
24    virNetworkCreate(network);  
25    virNetworkSetAutostart(network, 1);
```

```

25 // Save configuration
26 saveNetworksConfig();
27
28 result["success"] = true;
29 result["networkId"] = networkId;
30 return result;
31 }

```

Listing 4 – Création de réseau utilisateur

Génération XML :

```

1 std::string NetworkManager::generateUserNetworkXML(
2     const std::string& networkId,
3     const std::string& displayName,
4     const std::string& subnet) {
5
6     std::stringstream xml;
7     xml << "<network>"
8         << "<name>" << networkId << "</name>"
9         << "<bridge name='virbr-" << networkId.substr(0,8) << "'/>"
10        << "<forward mode='nat'/'>"
11        << "<ip address='" << subnet << ".1' netmask
12        << '=255.255.255.0'>"
13        << "<dhcp>"
14        << "<range start='" << subnet << ".100' "
15        << "end='" << subnet << ".200'/'>"
16        << "</dhcp>"
17        << "</ip>"
18        << "</network>";
19     return xml.str();
20 }

```

Listing 5 – XML pour réseau utilisateur

3.4 Flavours et Gestion des Ressources

Système de Flavours

Les flavours définissent des configurations prédéfinies de ressources avec tarification associée.

3.4.1 Définition des Flavours

Structure dans `flavor_manager.hpp` :

```

1 struct Flavor {
2     std::string id;
3     std::string name;
4     std::string description;
5     int vcpus;          // Nombre de vCPUs
6     int ram;            // RAM en MB

```

```

7   int disk;           // Disque en GB
8   int price;          // Prix en FCFA/mois
9   bool active;
10 };

```

Listing 6 – Structure Flavor

3.4.2 Flavours par Défaut

ID	Description	vCPU	RAM	Disk	Prix
nano	Entry level - Testing	1	512 MB	5 GB	800 FCFA
micro	Small apps	1	1 GB	10 GB	1500 FCFA
small	Light workloads	1	2 GB	15 GB	3500 FCFA
medium	Standard workloads	2	4 GB	20 GB	6500 FCFA
large	Heavy workloads	2	8 GB	40 GB	12000 FCFA
xlarge	Database & Analytics	4	16 GB	80 GB	22000 FCFA

TABLE 2 – Catalogue de Flavours

3.4.3 Validation des Ressources

```

1 json HostManager::checkResourceAvailability(int memory,
2                                           int cpu,
3                                           long long disk) {
4     json result;
5     result["available"] = false;
6
7     for (auto& [id, host] : hosts) {
8         if (!host.active) continue;
9
10        updateHostResources(host);
11
12        bool canAccommodate =
13            host.availableMemory >= (memory * 1024) &&
14            host.availableCPUs >= cpu &&
15            host.availableDisk >= disk;
16
17        if (canAccommodate) {
18            result["available"] = true;
19            result["selectedHost"] = id;
20            break;
21        }
22    }
23
24    return result;
25 }

```

Listing 7 – Vérification de disponibilité

3.5 Gestion Multi-Hôtes

Pool d'Hôtes KVM

Le système gère plusieurs hôtes KVM avec sélection automatique basée sur différentes stratégies.

3.5.1 Stratégies de Sélection d'Hôte

1. Least Used (Moins Chargé) - Stratégie par défaut

```
1 std::string HostManager::selectByLeastUsed(int mem, int cpu,
2                                           long long disk) {
3     std::string bestHost;
4     double bestScore = -1;
5
6     for (auto& [id, host] : hosts) {
7         if (!host.active) continue;
8
9         updateHostResources(host);
10
11         // Check sufficient resources
12         if (host.availableMemory < mem * 1024 ||
13             host.availableCPUs < cpu ||
14             host.availableDisk < disk) {
15             continue;
16         }
17
18         // Score based on free resources percentage
19         double memScore = (host.availableMemory /
20                             (double)host.totalMemory) * 100;
21         double cpuScore = (host.availableCPUs /
22                             (double)host.totalCPUs) * 100;
23         double diskScore = (host.availableDisk /
24                              (double)host.totalDisk) * 100;
25
26         // Weighted: 40% mem, 40% CPU, 20% disk
27         double score = memScore * 0.4 + cpuScore * 0.4 +
28                         diskScore * 0.2;
29
30         if (score > bestScore) {
31             bestScore = score;
32             bestHost = id;
33         }
34     }
35
36     return bestHost;
37 }
```

Listing 8 – Stratégie Least Used

2. Round Robin

```

1 std::string HostManager::selectByRoundRobin(int mem, int cpu,
2                                           long long disk) {
3     if (hosts.empty()) return "";
4
5     std::vector<std::string> hostIds;
6     for (const auto& [id, host] : hosts) {
7         if (host.active) hostIds.push_back(id);
8     }
9
10    size_t attempts = 0;
11    while (attempts < hostIds.size()) {
12        size_t currentIndex = (roundRobinIndex + attempts) %
13                               hostIds.size();
14        std::string hostId = hostIds[currentIndex];
15        auto& host = hosts[hostId];
16
17        updateHostResources(host);
18
19        if (host.availableMemory >= mem * 1024 &&
20            host.availableCPUs >= cpu &&
21            host.availableDisk >= disk) {
22
23            roundRobinIndex = (currentIndex + 1) % hostIds.size();
24            return hostId;
25        }
26        attempts++;
27    }
28
29    return "";
30 }

```

Listing 9 – Stratégie Round Robin

3. Best Fit

```

1 std::string HostManager::selectByBestFit(int mem, int cpu,
2                                           long long disk) {
3     std::string bestHost;
4     double bestFit = std::numeric_limits<double>::max();
5
6     for (auto& [id, host] : hosts) {
7         if (!host.active) continue;
8
9         updateHostResources(host);
10
11        if (host.availableMemory < mem * 1024 ||
12            host.availableCPUs < cpu ||
13            host.availableDisk < disk) {
14            continue;
15        }
16
17        // Calculate "waste" - leftover resources

```

```
18     double memWaste = (host.availableMemory / 1024.0 - mem) /
19                       (double)host.totalMemory;
20     double cpuWaste = (host.availableCPUs - cpu) /
21                      (double)host.totalCPUs;
22     double diskWaste = (host.availableDisk - disk) /
23                       (double)host.totalDisk;
24
25     double wasteScore = memWaste * 0.4 + cpuWaste * 0.4 +
26                       diskWaste * 0.2;
27
28     if (wasteScore < bestFit) {
29         bestFit = wasteScore;
30         bestHost = id;
31     }
32 }
33
34 return bestHost;
35 }
```

Listing 10 – Stratégie Best Fit

3.5.2 Autres Stratégies Possibles

1. **Priority-Based** : Hôtes avec priorité différente selon leur capacité
2. **Geographic** : Sélection basée sur la proximité géographique
3. **Cost-Optimized** : Minimisation des coûts opérationnels
4. **Power-Aware** : Réduction de la consommation électrique
5. **Load-Balanced** : Équilibrage de charge avec seuils

4 Partie 2 : Cluster Docker Swarm

4.1 Déploiement Automatique

Processus de Création

Le système déploie automatiquement un cluster Swarm avec le nombre souhaité de managers et workers.

4.1.1 Architecture du Cluster

```
1 json SwarmOperations::createSwarmCluster(
2     const std::string& clusterName,
3     const std::string& owner,
4     int numManagers,
5     int numWorkers) {
6
7     json result;
8 }
```

```
9      // 1. Create dedicated network for cluster
10     json networkResult = networkMgr->createSwarmNetwork(
11         clusterName, owner);
12
13     if (!networkResult["success"].get<bool>()) {
14         result["error"] = "Failed to create cluster network";
15         return result;
16     }
17
18     std::string networkId = networkResult["network"]["networkId"];
19
20     // 2. Deploy manager nodes
21     for (int i = 0; i < numManagers; i++) {
22         json vmConfig = {
23             {"hostname", clusterName + "-manager-" +
24                 std::to_string(i+1)},
25             {"owner", owner},
26             {"role", "manager"},
27             {"network", networkId},
28             {"flavor", "medium"},
29             {"baseImage", "ubuntu-22.04"}
30         };
31
32         json vmResult = deploySwarmNode(vmConfig);
33         if (!vmResult["success"].get<bool>()) {
34             // Rollback on failure
35             deleteCluster(clusterId);
36             return vmResult;
37         }
38     }
39
40     // 3. Deploy worker nodes
41     for (int i = 0; i < numWorkers; i++) {
42         json vmConfig = {
43             {"hostname", clusterName + "-worker-" +
44                 std::to_string(i+1)},
45             {"owner", owner},
46             {"role", "worker"},
47             {"network", networkId},
48             {"flavor", "small"},
49             {"baseImage", "ubuntu-22.04"}
50         };
51
52         deploySwarmNode(vmConfig);
53     }
54
55     // 4. Initialize Swarm on first manager
56     initializeSwarm(clusterName);
57
58     result["success"] = true;
59     result["clusterId"] = clusterId;
```

```
60     return result;  
61 }
```

Listing 11 – Création de cluster

4.1.2 Installation Docker via Cloud-Init

```
1  #cloud-config  
2  package_update: true  
3  package_upgrade: true  
4  
5  packages:  
6    - docker.io  
7    - docker-compose  
8  
9  runcmd:  
10   # Enable and start Docker  
11   - systemctl enable docker  
12   - systemctl start docker  
13  
14   # Add user to docker group  
15   - usermod -aG docker ubuntu  
16  
17   # Open Swarm ports  
18   - ufw allow 2377/tcp      # Cluster management  
19   - ufw allow 7946/tcp      # Node communication  
20   - ufw allow 7946/udp  
21   - ufw allow 4789/udp      # Overlay network  
22  
23   # Install Docker Compose  
24   - curl -L "https://github.com/docker/compose/releases/latest/  
    download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local  
    /bin/docker-compose  
25   - chmod +x /usr/local/bin/docker-compose
```

Listing 12 – Cloud-init pour Docker

4.1.3 Initialisation du Swarm

```
1  json SwarmOperations::initializeSwarmOnManager(  
2    const std::string& vmName,  
3    const std::string& managerIP) {  
4  
5    json result;  
6  
7    // Initialize swarm  
8    std::string initCmd = "docker swarm init --advertise-addr " +  
9                          managerIP;  
10   auto initResult = remoteExec->execute(initCmd);  
11 }
```

```
12  if (!initResult.success()) {
13      result["error"] = "Failed to initialize swarm";
14      return result;
15  }
16
17  // Get manager token
18  std::string managerTokenCmd =
19      "docker swarm join-token manager -q";
20  auto tokenResult = remoteExec->execute(managerTokenCmd);
21  std::string managerToken = tokenResult.output;
22
23  // Get worker token
24  std::string workerTokenCmd =
25      "docker swarm join-token worker -q";
26  auto workerTokenResult = remoteExec->execute(workerTokenCmd);
27  std::string workerToken = workerTokenResult.output;
28
29  result["success"] = true;
30  result["managerToken"] = managerToken;
31  result["workerToken"] = workerToken;
32
33  return result;
34 }
```

Listing 13 – Initialisation Swarm Manager

4.1.4 Jonction des Workers

```
1  json SwarmOperations::joinWorkerToSwarm(
2      const std::string& vmName,
3      const std::string& workerIP,
4      const std::string& managerIP,
5      const std::string& token) {
6
7      std::string joinCmd =
8          "docker swarm join --token " + token +
9          " " + managerIP + ":2377";
10
11      auto result = remoteExec->execute(joinCmd);
12
13      json response;
14      response["success"] = result.success();
15
16      return response;
17 }
```

Listing 14 – Ajout de worker au cluster

4.2 Exemple de Déploiement

Cluster 1 Manager + 2 Workers

Configuration recommandée pour démonstration

Nœud	Rôle	Flavor	IP
demo-manager-1	Manager	medium (2 vCPU, 4GB)	192.168.100.101
demo-worker-1	Worker	small (1 vCPU, 2GB)	192.168.100.102
demo-worker-2	Worker	small (1 vCPU, 2GB)	192.168.100.103

TABLE 3 – Configuration du Cluster de Démonstration

5 Partie 3 : Service PaaS

5.1 Catalogue d'Applications

Applications Disponibles

Déploiement en un clic de 8 applications populaires avec bases de données mutualisées.

5.1.1 Liste des Applications

Application	Description	Base
WordPress	CMS et blogging	MariaDB
Odoo	ERP/CRM	PostgreSQL
Mattermost	Communication d'équipe	PostgreSQL
OwnCloud	Stockage cloud	MariaDB
OnlyOffice	Suite bureautique	PostgreSQL
OpenLDAP	Annuaire LDAP	-
Moodle	Plateforme e-learning	MariaDB
PrestaShop	E-commerce	MariaDB

TABLE 4 – Catalogue d'Applications PaaS

5.2 Mutualisation des Bases de Données

Optimisation des Ressources

Une seule instance PostgreSQL et une seule MariaDB partagées par toutes les applications avec séparation logique.

5.2.1 Architecture de Mutualisation

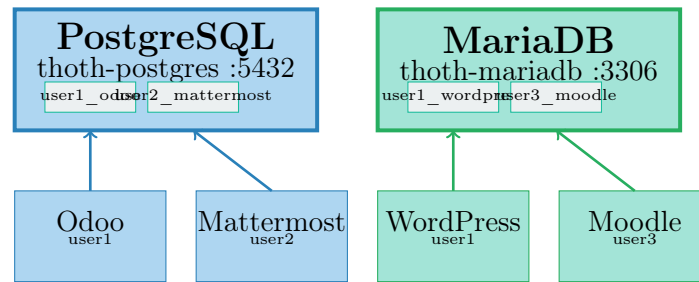


FIGURE 2 – Architecture de Mutualisation des Bases de Données

5.2.2 Création de Base de Données

```

1 json PaaSOperations::provisionDatabase(const std::string& username,
2                                     const std::string& appName,
3                                     const std::string& dbType) {
4     json result;
5
6     std::string dbName = username + "_" + appName;
7     std::string dbUser = dbName + "_user";
8     std::string dbPass = generateRandomPassword();
9
10    std::string script = "/var/lib/thoth-paas/manage-databases.sh";
11    std::string cmd;
12
13    if (dbType == "postgresql") {
14        cmd = script + " create-pg " + username + " " + appName;
15    } else if (dbType == "mysql") {
16        cmd = script + " create-mysql " + username + " " + appName;
17    }
18
19    auto execResult = remoteExec->execute(cmd);
20
21    if (execResult.success()) {
22        result["success"] = true;
23        result["database"] = {
24            {"host", dbType == "postgresql" ?
25             "thoth-postgres" : "thoth-mariadb"},
26            {"port", dbType == "postgresql" ? 5432 : 3306},
27            {"database", dbName},
28            {"username", dbUser},
29            {"password", dbPass}
30        };
31    }
32
33    return result;
34 }
  
```

Listing 15 – Provisionnement de base de données

5.2.3 Script de Gestion des Bases

Extrait de `manage-databases.sh` :

```
1 create-pg)
2     USERNAME=$2
3     APPNAME=$3
4     DB_NAME="${USERNAME}_${APPNAME}"
5     DB_USER="${DB_NAME}_user"
6     DB_PASS=$(openssl rand -base64 16)
7
8     # Create database and user
9     docker exec thoth-postgres psql -U postgres <<EOSQL
10     CREATE DATABASE ${DB_NAME};
11     CREATE USER ${DB_USER} WITH PASSWORD '${DB_PASS}';
12     GRANT ALL PRIVILEGES ON DATABASE ${DB_NAME} TO ${DB_USER};
13 EOSQL
14
15     echo "$DB_NAME|$DB_USER|$DB_PASS"
16     ;;
```

Listing 16 – Création base PostgreSQL

5.3 Déploiement d'Applications

5.3.1 Exemple : WordPress

```
1 json PaaSOperations::deployWordPress(const std::string& username,
2                                     const std::string& appName) {
3     json result;
4
5     // 1. Provision database
6     json dbResult = provisionDatabase(username, appName, "mysql");
7
8     // 2. Generate docker-compose
9     std::string composeFile = generateWordPressCompose(
10         username, appName, dbResult["database"]);
11
12     // 3. Deploy to Swarm
13     std::string stackName = username + "_" + appName;
14     std::string deployCmd =
15         "docker stack deploy -c " + composeFile + " " + stackName;
16
17     auto deployResult = remoteExec->execute(deployCmd);
18
19     result["success"] = deployResult.success();
20     result["url"] = "http://" + getNodeIP() + ":" +
21         std::to_string(assignedPort);
22
23     return result;
24 }
```

Listing 17 – Déploiement WordPress

```

1 version: '3.8'
2 services:
3   wordpress:
4     image: wordpress:latest
5     environment:
6       WORDPRESS_DB_HOST: thoth-mariadb
7       WORDPRESS_DB_NAME: ${DB_NAME}
8       WORDPRESS_DB_USER: ${DB_USER}
9       WORDPRESS_DB_PASSWORD: ${DB_PASS}
10    ports:
11      - "${PORT}:80"
12    networks:
13      - paas_network
14    deploy:
15      replicas: 1
16
17 networks:
18   paas_network:
19     external: true
20     name: thoth_paas_network

```

Listing 18 – Docker Compose WordPress

5.4 Système de Facturation

Modèle de Facturation

Facturation basée sur la consommation réelle des ressources (CPU, RAM, disque, durée).

5.4.1 Calcul des Coûts

```

1 struct BillingInfo {
2   double cpuHours;           // vCPU-heures
3   double ramGBHours;         // GB-heures
4   double diskGB;             // GB utilis s
5   double durationHours;      // Dur e totale
6
7   double calculateCost() const {
8     const double CPU_RATE = 10.0;      // FCFA/vCPU/heure
9     const double RAM_RATE = 5.0;       // FCFA/GB/heure
10    const double DISK_RATE = 0.5;       // FCFA/GB/mois
11
12    double cpuCost = cpuHours * CPU_RATE;
13    double ramCost = ramGBHours * RAM_RATE;
14    double diskCost = diskGB * DISK_RATE *
15      (durationHours / 720.0); // 720h = 1 mois

```

```

16         return cpuCost + ramCost + diskCost;
17     }
18 };
19
20
21 json getUserBilling(const std::string& username) {
22     json result;
23     double totalCost = 0.0;
24
25     // Get user's applications
26     auto apps = listUserApplications(username);
27
28     for (const auto& app : apps) {
29         BillingInfo billing = calculateAppBilling(app);
30         totalCost += billing.calculateCost();
31     }
32
33     result["username"] = username;
34     result["totalCost"] = totalCost;
35     result["currency"] = "FCFA";
36
37     return result;
38 }

```

Listing 19 – Calcul de facturation

5.4.2 Exemple de Facturation

Ressource	Consommation	Tarif	Coût
vCPU	2 vCPU × 720h	10 FCFA/vCPU/h	14,400 FCFA
RAM	4 GB × 720h	5 FCFA/GB/h	14,400 FCFA
Disque	20 GB × 1 mois	0.5 FCFA/GB/mois	10 FCFA
Total			28,810 FCFA

TABLE 5 – Exemple de Facturation Mensuelle

6 Partie 4 : Service SaaS

6.1 Plateforme Collaborative

Google Workspace Alternative

OwnCloud + OnlyOffice fournissent une suite complète de collaboration : stockage, partage, édition de documents.

6.1.1 Architecture SaaS

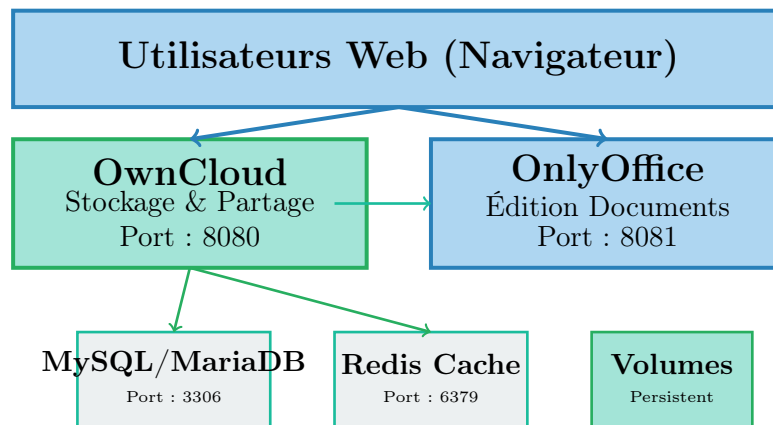


FIGURE 3 – Architecture SaaS - OwnCloud + OnlyOffice

6.1.2 Déploiement Docker Compose

Extrait de ownCloudSetup.sh :

```

1 version: '3.8'
2
3 services:
4   mysql:
5     image: mariadb:11
6     container_name: thoth-workspace-mysql
7     environment:
8       MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASS}
9       MYSQL_DATABASE: owncloud
10      MYSQL_USER: owncloud
11      MYSQL_PASSWORD: ${MYSQL_OWNCLOUD_PASS}
12     volumes:
13       - mysql_data:/var/lib/mysql
14     networks:
15       - workspace_network
16
17   redis:
18     image: redis:7-alpine
19     container_name: thoth-workspace-redis
20     volumes:
21       - redis_data:/data
22     networks:
23       - workspace_network
24
25   owncloud:
26     image: owncloud/server:latest
27     container_name: thoth-owncloud
28     depends_on:
29       - mysql
30       - redis
31     environment:

```

```
32     OWNCLOUD_DOMAIN: cloud.thoth.local
33     OWNCLOUD_DB_TYPE: mysql
34     OWNCLOUD_DB_HOST: mysql
35     OWNCLOUD_ADMIN_USERNAME: admin
36     OWNCLOUD_ADMIN_PASSWORD: ${ADMIN_PASS}
37     OWNCLOUD_REDIS_ENABLED: "true"
38     OWNCLOUD_REDIS_HOST: redis
39     volumes:
40     - owncloud_data:/mnt/data
41     ports:
42     - "8080:8080"
43     networks:
44     - workspace_network
45
46 onlyoffice:
47     image: onlyoffice/documentserver:latest
48     container_name: thoth-onlyoffice
49     environment:
50     JWT_ENABLED: "true"
51     JWT_SECRET: "thoth_jwt_secret_2024"
52     volumes:
53     - onlyoffice_data:/var/www/onlyoffice/Data
54     ports:
55     - "8081:80"
56     networks:
57     - workspace_network
58
59 networks:
60     workspace_network:
61     driver: bridge
62     name: thoth_workspace_network
```

Listing 20 – Docker Compose SaaS

6.1.3 Intégration OwnCloud + OnlyOffice

```
1  # Install OnlyOffice app in OwnCloud
2  docker exec thoth-owncloud occ market:install onlyoffice
3  docker exec thoth-owncloud occ app:enable onlyoffice
4
5  # Configure OnlyOffice integration
6  docker exec thoth-owncloud occ config:system:set \
7      onlyoffice DocumentServerUrl \
8      --value="http://thoth-onlyoffice/"
9
10 docker exec thoth-owncloud occ config:system:set \
11     onlyoffice jwt_secret \
12     --value="thoth_jwt_secret_2024"
```

Listing 21 – Configuration de l'intégration

6.2 Provisionnement d'Utilisateurs

```
1 #!/bin/bash
2 # provision-user.sh
3
4 USERNAME=$1
5 PASSWORD=$2
6 EMAIL=$3
7 QUOTA=${4:-10GB}
8
9 echo "Provisioning user: $USERNAME"
10
11 # Create user in OwnCloud
12 docker exec thoth-owncloud occ user:add "$USERNAME" \
13     --password-from-env \
14     --email="$EMAIL" <<< "$PASSWORD"
15
16 # Set storage quota
17 docker exec thoth-owncloud occ user:setting "$USERNAME" \
18     files quota "$QUOTA"
19
20 echo "User $USERNAME created with $QUOTA quota"
21 echo "Access: http://cloud.thoth.local:8080"
```

Listing 22 – Script de provisionnement

6.3 Facturation Stockage

```
1 #!/bin/bash
2 # billing-tracker.sh
3
4 OWNCLOUD_DATA="/var/lib/thoth-saas/owncloud"
5 BILLING_RATE=0.10 # $ per GB per month
6
7 calculate_storage() {
8     USAGE_BYTES=$(du -sb "$OWNCLOUD_DATA" | cut -f1)
9     USAGE_GB=$(echo "scale=2; $USAGE_BYTES/1024/1024/1024" | bc)
10    MONTHLY_COST=$(echo "scale=2; $USAGE_GB * $BILLING_RATE" | bc)
11
12    cat > billing.json <<JSON
13 {
14     "storage_gb": ${USAGE_GB},
15     "rate_per_gb": ${BILLING_RATE},
16     "monthly_cost": ${MONTHLY_COST},
17     "timestamp": "$(date -Iseconds)"
18 }
19 JSON
20
21 echo "Storage: ${USAGE_GB} GB"
22 echo "Cost: \${MONTHLY_COST}/month"
23 }
```

Listing 23 – Calcul facturation stockage

Utilisateur	Stockage	Tarif	Coût
alice	5.2 GB	0.10 \$/GB	0.52 \$
bob	12.8 GB	0.10 \$/GB	1.28 \$
charlie	3.1 GB	0.10 \$/GB	0.31 \$
Total	21.1 GB		2.11 \$

TABLE 6 – Exemple de Facturation Stockage

7 Documentation API

7.1 Authentification

Token-Based Authentication

L'API utilise des tokens de session avec expiration d'1 heure.

7.1.1 Login

```
1 POST /api/auth/login
2 Content-Type: application/json
3
4 {
5   "username": "alice",
6   "password": "secret123"
7 }
8
9 Response:
10 {
11   "success": true,
12   "token": "a1b2c3d4e5f6...",
13   "user": {
14     "username": "alice",
15     "role": "user",
16     "email": "alice@example.com"
17   }
18 }
```

Listing 24 – Endpoint de connexion

7.2 Gestion des VMs

7.2.1 Déployer une VM

```
1 POST /api/vms/deploy
2 Authorization: Bearer {token}
3 Content-Type: application/json
4
5 {
6   "hostname": "webserver",
7   "flavor": "medium",
8   "baseImage": "ubuntu-22.04",
9   "network": "user_network_id",
10  "username": "ubuntu",
11  "password": "mypass123"
12 }
13
14 Response:
15 {
16   "success": true,
17   "vm": {
18     "name": "alice__webserver__1234567890",
19     "status": "running",
20     "ip": "192.168.100.50"
21   }
22 }
```

Listing 25 – Création de VM

7.2.2 Lister les VMs

```
1 GET /api/vms/list
2 Authorization: Bearer {token}
3
4 Response:
5 {
6   "success": true,
7   "vms": [
8     {
9       "name": "alice__webserver__1234567890",
10      "displayName": "webserver",
11      "status": "running",
12      "ip": "192.168.100.50",
13      "vcpus": 2,
14      "memory": 4096,
15      "created": 1704067200000
16    }
17  ]
18 }
```

Listing 26 – Liste des VMs utilisateur

7.3 Gestion Swarm

7.3.1 Créer un Cluster

```
1 POST /api/swarm/create
2 Authorization: Bearer {token}
3 Content-Type: application/json
4
5 {
6   "clusterName": "production",
7   "numManagers": 1,
8   "numWorkers": 2
9 }
10
11 Response:
12 {
13   "success": true,
14   "cluster": {
15     "clusterId": "cluster_abc123",
16     "clusterName": "production",
17     "nodes": [
18       {
19         "name": "production-manager-1",
20         "role": "manager",
21         "ip": "192.168.100.101"
22       },
23       {
24         "name": "production-worker-1",
25         "role": "worker",
26         "ip": "192.168.100.102"
27       }
28     ]
29   }
30 }
```

Listing 27 – Création cluster Swarm

7.4 Déploiement PaaS

```
1 POST /api/paas/deploy
2 Authorization: Bearer {token}
3 Content-Type: application/json
4
5 {
6   "appType": "wordpress",
7   "appName": "myblog",
8   "cluster": "production"
9 }
10
11 Response:
12 {
13   "success": true,
14   "application": {
```

```
15     "appId": "app_xyz789",
16     "appName": "myblog",
17     "type": "wordpress",
18     "url": "http://192.168.100.101:8001",
19     "database": {
20         "host": "thoth-mariadb",
21         "database": "alice_myblog",
22         "username": "alice_myblog_user"
23     }
24 }
25 }
```

Listing 28 – Déploiement application

8 Guide de Déploiement

8.1 Prérequis

1. **Système** : Ubuntu 22.04 LTS ou supérieur
2. **CPU** : Support de virtualisation (Intel VT-x / AMD-V)
3. **RAM** : Minimum 16 GB recommandé
4. **Disque** : 100 GB minimum
5. **Réseau** : Connexion Internet

8.2 Installation

8.2.1 1. Cloner le Repository

```
1 git clone https://github.com/votre-org/thoth-cloud.git
2 cd thoth-cloud
```

Listing 29 – Clonage du projet

8.2.2 2. Setup KVM Host

```
1 # Run setup script as root
2 sudo bash scripts/setup_remote.sh
3
4 # This installs:
5 # - KVM, QEMU, libvirt
6 # - Creates 'ups' user
7 # - Configures passwordless sudo
8 # - Sets up firewall rules
```

Listing 30 – Configuration hôte KVM

8.2.3 3. Compiler le Backend

```
1 # Install dependencies
2 sudo bash scripts/setup-libs.sh
3
4 # Create build directory
5 mkdir build && cd build
6
7 # Compile
8 cmake ..
9 make -j$(nproc)
10
11 # Binary will be at: build/thoth_cloud
```

Listing 31 – Compilation C++

8.2.4 4. Configurer l'Environnement

```
1 # Create config directory
2 sudo mkdir -p /var/lib/thoth-cloud
3
4 # Create .env file
5 cat > /var/lib/thoth-cloud/.env <<EOF
6 API_PORT=3000
7 HOSTS=qemu:///system
8 EOF
```

Listing 32 – Configuration .env

8.2.5 5. Initialiser les Bases de Données

```
1 sudo bash scripts/paas-db-setup.sh
2
3 # This creates:
4 # - thoth-postgres container
5 # - thoth-mariadb container
6 # - Database management scripts
```

Listing 33 – Setup bases partagées

8.2.6 6. Démarrer le Backend

```
1 # Run backend
2 sudo ./build/thoth_cloud
3
4 # Server starts on http://localhost:3000
5 # Check logs in /var/log/thoth-cloud/
```

Listing 34 – Lancement du serveur

8.2.7 7. Créer un Utilisateur Admin

```
1 # Compile and run admin creator
2 g++ -o create_admin src/create_admin.cpp \
3     -lvirt -lcrypto $(pkg-config --cflags --libs)
4
5 sudo ./create_admin
6
7 # Default admin credentials:
8 # Username: admin
9 # Password: admin123
```

Listing 35 – Création admin

8.3 Multi-Hôtes (Optionnel)

```
1 # On remote host, run:
2 sudo bash scripts/set-remote.sh
3
4 # Get remote host IP
5 REMOTE_IP=$(hostname -I | awk '{print $1}')
6
7 # On backend server, generate SSH key
8 ssh-keygen -t rsa -b 4096 -f ~/.ssh/thoth_kvm_key -N ''
9
10 # Copy key to remote
11 ssh-copy-id -i ~/.ssh/thoth_kvm_key.pub vps@$REMOTE_IP
12
13 # Test connection
14 virsh -c qemu+ssh://vps@$REMOTE_IP/system list
15
16 # Add to configuration
17 # Edit /var/lib/thoth-cloud/.env:
18 # HOSTS=qemu:///system,qemu+ssh://vps@$REMOTE_IP/system
```

Listing 36 – Ajout hôte distant

8.4 Vérification

```
1 # Test API health
2 curl http://localhost:3000/api/health
3
4 # Login
5 curl -X POST http://localhost:3000/api/auth/login \
6     -H "Content-Type: application/json" \
7     -d '{"username":"admin","password":"admin123"}'
8
9 # List VMs
10 TOKEN="your_token_here"
11 curl http://localhost:3000/api/vms/list \
```

```
12 -H "Authorization: Bearer $TOKEN"
```

Listing 37 – Tests de vérification

9 Limites et Améliorations

9.1 Limites Connues

1. Sécurité :

- Tokens stockés en mémoire (perdus au redémarrage)
- Pas de rotation automatique des secrets
- Authentification basique (pas de 2FA)

2. Scalabilité :

- Backend mono-instance
- Pas de load balancing natif
- Limite de 20 hôtes KVM

3. Monitoring :

- Métriques limitées
- Pas d'alerting automatique
- Logs non centralisés

4. Réseau :

- Gestion basique des sous-réseaux
- Pas de VLAN support
- Firewall rules manuelles

9.2 Améliorations Futures

Roadmap

Améliorations prévues pour les prochaines versions

9.2.1 Court Terme (3 mois)

1. Authentification :

- OAuth2/OpenID Connect
- LDAP/Active Directory integration
- Two-factor authentication (2FA)
- Session persistence (Redis)

2. Monitoring :

- Intégration Prometheus/Grafana
- Métriques détaillées (CPU, RAM, réseau, I/O)
- Alerting (email, Slack, webhook)
- Dashboard temps réel

3. Backups :

- Snapshots automatiques
- Backup incrémental
- Restauration rapide
- Réplication cross-host

9.2.2 Moyen Terme (6 mois)**1. HA et Clustering :**

- Backend multi-instances
- Load balancer (HAProxy/Nginx)
- Failover automatique
- Shared storage (Ceph/GlusterFS)

2. Networking Avancé :

- SDN (Software-Defined Networking)
- VLAN support
- Firewall programmable
- VPN inter-sites

3. Kubernetes :

- Support K8s en plus de Swarm
- Helm charts
- Operators personnalisés
- Multi-cluster management

9.2.3 Long Terme (12 mois)**1. AI/ML Integration :**

- Auto-scaling prédictif
- Anomaly detection
- Resource optimization
- Cost forecasting

2. Marketplace :

- Catalogue d'images personnalisées
- Templates d'infrastructure
- Partage communautaire
- Validation automatique

3. Compliance :

- Audit trails
- GDPR compliance tools
- Encryption at rest
- Security scanning

10 Conclusion

Réalisations

THOTH CLOUD implémente avec succès une plateforme cloud complète intégrant IaaS, PaaS et SaaS.

10.1 Objectifs Atteints

1. IaaS Multi-Tenant :

- Isolation utilisateurs complète
- Gestion multi-hôtes avec 3 stratégies
- Système de flavors avec pricing
- Réseaux dédiés par utilisateur

2. PaaS Docker Swarm :

- Déploiement automatique de clusters
- 8 applications en catalogue
- Bases de données mutualisées
- Facturation à la consommation

3. SaaS Collaboratif :

- OwnCloud + OnlyOffice intégrés
- Alternative Google Workspace
- Facturation au stockage
- Multi-utilisateurs

10.2 Compétences Développées

- Architecture cloud multi-services
- Programmation système en C++
- Virtualisation KVM/libvirt avancée
- Orchestration de conteneurs
- Gestion de réseaux virtuels
- Développement d'API REST
- DevOps et automatisation

10.3 Perspectives

THOTH CLOUD constitue une base solide pour un cloud de production. Les améliorations futures permettront d'atteindre la maturité d'un cloud professionnel tout en conservant la simplicité d'utilisation.

THOTH CLOUD
Cloud Computing for Everyone

A Structure du Code

A.1 Organisation des Fichiers

```
1 thoth-cloud/  
2     include/                                # Header files (.hpp)  
3         baseimage_manager.hpp  
4         config_manager.hpp  
5         cors.hpp  
6         flavor_manager.hpp  
7         host_manager.hpp  
8         network_manager.hpp  
9         paas_operations.hpp  
10        remote_executor.hpp  
11        routes.hpp  
12        swarm_operations.hpp  
13        user_operations.hpp  
14        utils.hpp  
15        validation.hpp  
16        vm_operations.hpp  
17    src/                                    # Implementation files (.cpp)  
18        baseimage_manager.cpp  
19        config_manager.cpp  
20        cors.cpp  
21        flavor_manager.cpp  
22        host_manager.cpp  
23        main.cpp  
24        network_manager.cpp  
25        paas_operations.cpp  
26        remote_executor.cpp  
27        resource_logger.cpp  
28        routes.cpp  
29        routes2.cpp  
30        swarm_operations.cpp  
31        user_operations.cpp  
32        utils.cpp  
33        validation.cpp  
34        vm_operations.cpp  
35    scripts/                               # Deployment scripts  
36        general.sh  
37        setup_remote.sh  
38        paas-db-setup.sh  
39        ownCloudSetup.sh  
40        setup-novnc.sh  
41    build/                                # Compiled binaries
```

Listing 38 – Arborescence du projet

B Configuration Système

B.1 Fichiers de Configuration

B.1.1 /var/lib/thoth-cloud/.env

```
1 API_PORT=3000
2 HOSTS=qemu:///system,qemu+ssh://vps@192.168.1.100/system
3 BASE_IMAGE_DIR=/var/lib/libvirt/images/baseimg
4 CLOUD_INIT_DIR=/var/lib/libvirt/images/cloud-init-iso
```

Listing 39 – Fichier d'environnement

B.1.2 /var/lib/thoth-cloud/hosts.json

```
1 {
2   "hosts": [
3     {
4       "id": "localhost",
5       "uri": "qemu:///system",
6       "hostname": "localhost"
7     },
8     {
9       "id": "kvm-node-1",
10      "uri": "qemu+ssh://vps@192.168.1.100/system",
11      "hostname": "kvm-node-1"
12    }
13  ]
14 }
```

Listing 40 – Configuration des hôtes

C Commandes Utiles

C.1 Gestion des VMs

```
1 # List all VMs
2 virsh list --all
3
4 # Start VM
5 virsh start vm_name
6
7 # Shutdown VM
8 virsh shutdown vm_name
9
10 # Force stop
11 virsh destroy vm_name
12
13 # Delete VM
14 virsh undefine vm_name --remove-all-storage
15
16 # Get VM info
```

```
17 virsh dominfo vm_name
18
19 # Console access
20 virsh console vm_name
```

Listing 41 – Commandes virsh

C.2 Gestion Docker Swarm

```
1 # Initialize swarm
2 docker swarm init --advertise-addr <IP>
3
4 # Join as worker
5 docker swarm join --token <TOKEN> <MANAGER_IP>:2377
6
7 # List nodes
8 docker node ls
9
10 # Deploy stack
11 docker stack deploy -c compose.yml stack_name
12
13 # List services
14 docker service ls
15
16 # Scale service
17 docker service scale service_name=5
18
19 # Remove stack
20 docker stack rm stack_name
```

Listing 42 – Commandes Swarm